V Sunil Kumar

192425292

MLA0304

UNIT 1: FOUNDATIONS

EXPERIMENT 1:

CODE:

```
import numpy as np

import random


# --- MDP Environment ---

class CleaningRobotMDP:

    def __init__(self, size=5):

        self.size = size

        self.grid = np.zeros((size, size))


        # Rewards: +1 for dirt, -1 for obstacle, 0 for empty

        self.grid[0, 4] = +1

        self.grid[2, 2] = +1

        self.grid[4, 3] = +1

        self.grid[1, 2] = -1

        self.grid[2, 3] = -1

        self.grid[3, 1] = -1
```

```
    self.start = (0, 0)
    self.state = self.start
    self.total_reward = 0
    self.actions = ['UP', 'DOWN', 'LEFT', 'RIGHT']

def reset(self):
    self.state = self.start
    self.total_reward = 0
    return self.state

def step(self, action):
    x, y = self.state
    if action == 'UP': x -= 1
    elif action == 'DOWN': x += 1
    elif action == 'LEFT': y -= 1
    elif action == 'RIGHT': y += 1

    # Check boundaries
    if x < 0 or y < 0 or x >= self.size or y >= self.size:
        reward = -1  # wall hit
        x, y = self.state
    else:
        reward = self.grid[x, y]
```

```
            if reward == +1:  # clean dirt
                self.grid[x, y] = 0

        self.state = (x, y)
        self.total_reward += reward
        return self.state, reward

# --- Policies ---
def random_policy(env, steps=20):
    print("\n--- Random Policy ---")
    env.reset()
    for _ in range(steps):
        action = random.choice(env.actions)
        state, reward = env.step(action)
        print(f"Action: {action} | State: {state} | Reward: {reward}")
    print(f"Total Reward: {env.total_reward}\n")

def greedy_policy(env, steps=20):
    print("\n--- Greedy Policy ---")
    env.reset()
    for _ in range(steps):
        x, y = env.state
        # Move towards bottom-right corner (simple greedy heuristic)
        if x < env.size - 1: action = 'DOWN'
```

```
        elif y < env.size - 1: action = 'RIGHT'
        else: action = random.choice(env.actions)
        state, reward = env.step(action)
        print(f"Action: {action} | State: {state} | Reward: {reward}")
    print(f"Total Reward: {env.total_reward}\n")

# --- Run Simulation ---
env = CleaningRobotMDP()
random_policy(env)
greedy_policy(env)
```

OUTPUT:

```
--- Random Policy ---
Action: UP | State: (0, 0) | Reward: -1
Action: LEFT | State: (0, 0) | Reward: -1
Action: UP | State: (0, 0) | Reward: -1
Action: LEFT | State: (0, 0) | Reward: -1
Action: UP | State: (0, 0) | Reward: -1
Action: LEFT | State: (0, 0) | Reward: -1
Action: LEFT | State: (0, 0) | Reward: -1
Action: DOWN | State: (1, 0) | Reward: 0.0
Action: DOWN | State: (2, 0) | Reward: 0.0
Action: DOWN | State: (3, 0) | Reward: 0.0
Action: UP | State: (2, 0) | Reward: 0.0
Action: LEFT | State: (2, 0) | Reward: -1
Action: DOWN | State: (3, 0) | Reward: 0.0
Action: RIGHT | State: (3, 1) | Reward: -1.0
Action: RIGHT | State: (3, 2) | Reward: 0.0
Action: UP | State: (2, 2) | Reward: 1.0
Action: RIGHT | State: (2, 3) | Reward: -1.0
Action: RIGHT | State: (2, 4) | Reward: 0.0
Action: UP | State: (1, 4) | Reward: 0.0
Action: UP | State: (0, 4) | Reward: 1.0
Total Reward: -8.0
```

```
--- Greedy Policy ---
Action: DOWN | State: (1, 0) | Reward: 0.0
Action: DOWN | State: (2, 0) | Reward: 0.0
Action: DOWN | State: (3, 0) | Reward: 0.0
Action: DOWN | State: (4, 0) | Reward: 0.0
Action: RIGHT | State: (4, 1) | Reward: 0.0
Action: RIGHT | State: (4, 2) | Reward: 0.0
Action: RIGHT | State: (4, 3) | Reward: 1.0
Action: RIGHT | State: (4, 4) | Reward: 0.0
Action: DOWN | State: (4, 4) | Reward: -1
Action: UP | State: (3, 4) | Reward: 0.0
Action: DOWN | State: (4, 4) | Reward: 0.0
Action: UP | State: (3, 4) | Reward: 0.0
Action: DOWN | State: (4, 4) | Reward: 0.0
Action: RIGHT | State: (4, 4) | Reward: -1
Action: DOWN | State: (4, 4) | Reward: -1
Action: RIGHT | State: (4, 4) | Reward: -1
Action: RIGHT | State: (4, 4) | Reward: -1
Action: RIGHT | State: (4, 4) | Reward: -1
Action: RIGHT | State: (4, 4) | Reward: -1
Action: UP | State: (3, 4) | Reward: 0.0
Total Reward: -6.0
```

EXPERIMENT 2:.

CODE:

```
import numpy as np


# --- Environment setup ---
class WarehouseRobot:
    def __init__(self, size=4):
        self.size = size
        self.states = [(i, j) for i in range(size) for j in range(size)]
        self.actions = ['UP', 'DOWN', 'LEFT', 'RIGHT']
        self.goal = (3, 3)
        self.items = [(1, 1), (2, 2)]
        self.obstacles = [(1, 2), (2, 1)]

    def get_reward(self, state):
        if state in self.items:
            return 2   # picking an item
        elif state == self.goal:
            return 5   # reaching goal
        elif state in self.obstacles:
            return -2  # hitting obstacle
        else:
            return 0   # empty cell

    def next_state(self, state, action):
        x, y = state
        if action == 'UP': x -= 1
        elif action == 'DOWN': x += 1
        elif action == 'LEFT': y -= 1
        elif action == 'RIGHT': y += 1

        # Boundary conditions
        if x < 0 or y < 0 or x >= self.size or y >= self.size:
            return state  # stay in same place
        return (x, y)
```

```
# --- Policy Evaluation ---
def policy_evaluation(env, policy, gamma=0.9, theta=1e-4):
    V = {s: 0 for s in env.states}  # initial value function

    while True:
        delta = 0
        for s in env.states:
            v = V[s]
            new_v = 0
            for a, prob in policy[s].items():
                next_s = env.next_state(s, a)
                reward = env.get_reward(next_s)
                new_v += prob * (reward + gamma * V[next_s])
            V[s] = new_v
            delta = max(delta, abs(v - new_v))
        if delta < theta:
            break
    return V



# --- Define Environment ---
env = WarehouseRobot(size=4)

# --- Define a uniform random policy ---
policy = {
    s: {a: 0.25 for a in env.actions} for s in env.states
}

# --- Evaluate Policy ---
values = policy_evaluation(env, policy)

# --- Display Result ---
print("\nValue Function (after policy evaluation):")
for i in range(env.size):
    for j in range(env.size):
        print(f"{values[(i, j)]:6.2f}", end="  ")
```

print()

OUTPUT:

```
Value Function (after policy evaluation):
  1.37    1.67    1.12    1.56
  1.67    1.04    2.87    2.68
  1.12    2.87    3.45    7.03
  1.56    2.68    7.03   10.29
```

EXPERIMENT 3:

An online retailer uses a multi-armed bandit approach to set prices dynamically. Simulate different pricing strategies using epsilon-greedy, UCB, and Thompson Sampling. Write a Python script to compare which strategy maximizes revenue over a series of pricing decisions.

CODE:

```
import numpy as np
import random
from math import sqrt, log

# ----------------------------------------
# Pricing arms (Price points)
# Arm = a price, Reward = revenue
# ----------------------------------------
prices = [50, 60, 70, 80]       # example prices
true_conversion = [0.15, 0.12, 0.10, 0.07]  # probability customer buys
n_arms = len(prices)
rounds = 5000



# ----------------------------------------
# Helper: simulate revenue for an arm
# ----------------------------------------
def get_reward(arm):
    if random.random() < true_conversion[arm]:
        return prices[arm]   # customer buys → revenue = price
    else:
        return 0             # customer doesn't buy



# ----------------------------------------
# 1. EPSILON-GREEDY
# ----------------------------------------
def epsilon_greedy(epsilon=0.1):
    rewards = np.zeros(n_arms)
    counts = np.zeros(n_arms)
    total_reward = 0
```

```
    for t in range(rounds):
        if random.random() < epsilon:
            arm = random.randint(0, n_arms - 1)
        else:
            arm = np.argmax(rewards / (counts + 1e-5))

        r = get_reward(arm)
        counts[arm] += 1
        rewards[arm] += r
        total_reward += r

    return total_reward


# ---------------------------------------
# 2. UCB (Upper Confidence Bound)
# ---------------------------------------
def ucb():
    rewards = np.zeros(n_arms)
    counts = np.ones(n_arms)
    total_reward = 0

    for t in range(1, rounds + 1):
        ucb_values = rewards / counts + np.sqrt(2 * np.log(t) / counts)
        arm = np.argmax(ucb_values)

        r = get_reward(arm)
        counts[arm] += 1
        rewards[arm] += r
        total_reward += r

    return total_reward


# ---------------------------------------
# 3. THOMPSON SAMPLING
```

```
# ------------------------------------
def thompson_sampling():
    alpha = np.ones(n_arms)
    beta = np.ones(n_arms)
    total_reward = 0

    for _ in range(rounds):
        samples = [np.random.beta(alpha[i], beta[i]) for i in range(n_arms)]
        arm = np.argmax(samples)

        r = get_reward(arm)
        total_reward += r

        if r > 0:
            alpha[arm] += 1
        else:
            beta[arm] += 1

    return total_reward



# ------------------------------------
# Run all algorithms
# ------------------------------------
reward_eps = epsilon_greedy()
reward_ucb = ucb()
reward_ts = thompson_sampling()

print("Total Revenue Comparison:")
print("Epsilon-Greedy:", reward_eps)
print("UCB:", reward_ucb)
print("Thompson Sampling:", reward_ts)
```

OUTPUT:

```
Total Revenue Comparison:
Epsilon-Greedy: 37080
UCB: 34300
Thompson Sampling: 38180
```

EXPERIMENT 4:

A delivery drone needs to find the shortest path from a warehouse to multiple delivery points in a city represented as a grid. Implement a Policy Iteration algorithm using Dynamic Programming to compute the optimal routing policy for the drone. The grid consists of free cells, obstacles, and delivery points with rewards. Use Python to evaluate and improve the policy until convergence.

CODE:

```
import numpy as np

# ------------------------------------------------------
# 4x4 Grid City:
# W = warehouse (start)
# D = delivery point (+5)
# X = obstacle (-5)
# 0 = normal cell
# ------------------------------------------------------
grid = np.array([
    ['W', '0', '0', 'D'],
    ['0', 'X', '0', '0'],
    ['0', '0', '0', 'D'],
    ['0', '0', '0', '0']
])

# Rewards for grid cells
reward_map = {
    'W': 0,
    '0': 0,
    'D': 5,
    'X': -5
}

# All possible actions
actions = {
    'U': (-1, 0),
    'D': (1, 0),
    'L': (0, -1),
```

```
    'R': (0, 1)
}

gamma = 0.9   # discount factor
theta = 1e-4  # threshold for convergence

rows, cols = grid.shape

# Initial policy (randomly choose 'U' for all states)
policy = {(i, j): 'U' for i in range(rows) for j in range(cols)}

# Initialize value function
V = np.zeros((rows, cols))



def is_valid(i, j):
    return 0 <= i < rows and 0 <= j < cols



# ----------------------------------------------------------
# POLICY EVALUATION
# ----------------------------------------------------------
def policy_evaluation():
    global V
    while True:
        delta = 0
        new_V = np.copy(V)

        for i in range(rows):
            for j in range(cols):
                a = policy[(i, j)]
                di, dj = actions[a]
                ni, nj = i + di, j + dj

                if is_valid(ni, nj):
                    reward = reward_map[grid[ni][nj]]
                    new_V[i][j] = reward + gamma * V[ni][nj]
```

```
            else:
                # hitting wall = penalty
                new_V[i][j] = -5 + gamma * V[i][j]

            delta = max(delta, abs(V[i][j] - new_V[i][j]))

        V = new_V

        if delta < theta:
            break



# ---------------------------------------------------------
# POLICY IMPROVEMENT
# ---------------------------------------------------------
def policy_improvement():
    stable = True

    for i in range(rows):
        for j in range(cols):
            old_action = policy[(i, j)]

            best_value = -999
            best_action = old_action

            for a, (di, dj) in actions.items():
                ni, nj = i + di, j + dj

                if is_valid(ni, nj):
                    reward = reward_map[grid[ni][nj]]
                    value = reward + gamma * V[ni][nj]
                else:
                    value = -5 + gamma * V[i][j]

                if value > best_value:
                    best_value = value
                    best_action = a
```

```
            policy[(i, j)] = best_action

            if best_action != old_action:
                stable = False

    return stable


# ----------------------------------------------------
# POLICY ITERATION
# ----------------------------------------------------
while True:
    policy_evaluation()
    if policy_improvement():
        break


# Print results
print("Final Optimal Value Function:")
print(V)

print("\nOptimal Policy:")
for i in range(rows):
    row_policy = ''
    for j in range(cols):
        row_policy += policy[(i, j)] + ' '
    print(row_policy)
```

OUTPUT:

```
Final Optimal Value Function:
[[21.31557533 23.68397259 26.31557533 23.68397259]
 [19.18397259 21.31557533 23.68397259 26.31557533]
 [21.31557533 23.68397259 26.31557533 23.68397259]
 [19.18397259 21.31557533 23.68397259 26.31557533]]

Optimal Policy:
R R R D
U U U U
R R R U
U U U U
```

EXPERIMENT 5:

In a taxi dispatching system, define an MDP where states are locations, actions are moving directions, and rewards are based on reaching pick-up points quickly. Write a Python program to use value iteration to find the optimal dispatch policy.

CODE:

```
import numpy as np

# Grid size (3x3 city)
grid_size = 3

# Rewards
pickup = (2, 2)  # pickup point
R = np.zeros((grid_size, grid_size))
R[pickup] = 10   # reward for reaching pickup

# Actions: Up, Down, Left, Right
actions = {
    0: (-1, 0),
    1: (1, 0),
    2: (0, -1),
    3: (0, 1)
}

gamma = 0.9
theta = 0.0001

def is_valid(x, y):
    return 0 <= x < grid_size and 0 <= y < grid_size

def value_iteration():
    V = np.zeros((grid_size, grid_size))
    policy = np.zeros((grid_size, grid_size), dtype=int)
```

```
    while True:
        delta = 0
        for i in range(grid_size):
            for j in range(grid_size):
                values = []
                for a, (dx, dy) in actions.items():
                    nx, ny = i + dx, j + dy
                    if not is_valid(nx, ny):
                        nx, ny = i, j  # stay in place if invalid
                    values.append(R[nx, ny] + gamma * V[nx, ny])

                max_value = max(values)
                delta = max(delta, abs(max_value - V[i, j]))
                V[i, j] = max_value
                policy[i, j] = np.argmax(values)

        if delta < theta:
            break

    return V, policy

V, policy = value_iteration()

print("Optimal Value Function:\n", V)
print("\nOptimal Policy (0=Up,1=Down,2=Left,3=Right):\n", policy)
```

OUTPUT:

```
Optimal Value Function:
 [[72.89916648 80.99916648 89.99916648]
 [80.99916648 89.99916648 99.99916648]
 [89.99916648 99.99916648 99.99916648]]

Optimal Policy (0=Up,1=Down,2=Left,3=Right):
 [[1 1 1]
 [1 1 1]
 [3 3 1]]
```

EXPERIMENT 6:

An online platform uses bandit algorithms to decide which advertisements to show to users. Implement epsilon-greedy, UCB, and Thompson Sampling algorithms. Use a Python script to determine which algorithm results in the highest click-through rate over time.

CODE:

```
import numpy as np
import math
import random

# True click-through rates of ads
true_ctr = [0.05, 0.08, 0.12, 0.03]   # 4 ads

num_ads = len(true_ctr)
rounds = 5000

# Tracking results
eps_rewards = np.zeros(num_ads)
eps_counts = np.zeros(num_ads)

ucb_rewards = np.zeros(num_ads)
ucb_counts = np.zeros(num_ads)

ts_success = np.zeros(num_ads)
ts_failure = np.zeros(num_ads)

# ---- EPSILON GREEDY ----
epsilon = 0.1
eps_total_reward = 0

for t in range(rounds):
    if random.random() < epsilon:
        ad = random.randint(0, num_ads - 1)
    else:
        ad = np.argmax(eps_rewards / (eps_counts + 1))

    click = 1 if random.random() < true_ctr[ad] else 0
```

```
    eps_counts[ad] += 1
    eps_rewards[ad] += click
    eps_total_reward += click


# ---- UCB ----
ucb_total_reward = 0

for t in range(1, rounds + 1):
    ucb_values = []
    for ad in range(num_ads):
        if ucb_counts[ad] == 0:
            ucb_values.append(float('inf'))
        else:
            avg = ucb_rewards[ad] / ucb_counts[ad]
            bonus = math.sqrt(2 * math.log(t) / ucb_counts[ad])
            ucb_values.append(avg + bonus)
    ad = np.argmax(ucb_values)

    click = 1 if random.random() < true_ctr[ad] else 0
    ucb_counts[ad] += 1
    ucb_rewards[ad] += click
    ucb_total_reward += click


# ---- THOMPSON SAMPLING ----
ts_total_reward = 0

for t in range(rounds):
    samples = [np.random.beta(ts_success[i] + 1, ts_failure[i] + 1)
               for i in range(num_ads)]
    ad = np.argmax(samples)

    click = 1 if random.random() < true_ctr[ad] else 0

    if click == 1:
        ts_success[ad] += 1
```

```
    else:
        ts_failure[ad] += 1

    ts_total_reward += click



print("Epsilon-Greedy CTR:", eps_total_reward / rounds)
print("UCB CTR:", ucb_total_reward / rounds)
print("Thompson Sampling CTR:", ts_total_reward / rounds)

best = max(eps_total_reward, ucb_total_reward, ts_total_reward)
print("\nBest Algorithm:",
      "Thompson Sampling" if best == ts_total_reward
      else "UCB" if best == ucb_total_reward
      else "Epsilon-Greedy")
```

OUTPUT:

```
Epsilon-Greedy CTR: 0.1028
UCB CTR: 0.0918
Thompson Sampling CTR: 0.1138

Best Algorithm: Thompson Sampling
```

EXPERIMENT 7:

A delivery robot operates in a warehouse with predefined delivery points. Using Bellman equations, compute the state-value function for navigating to each delivery point. Implement this in Python and visualize the value function for different policies.

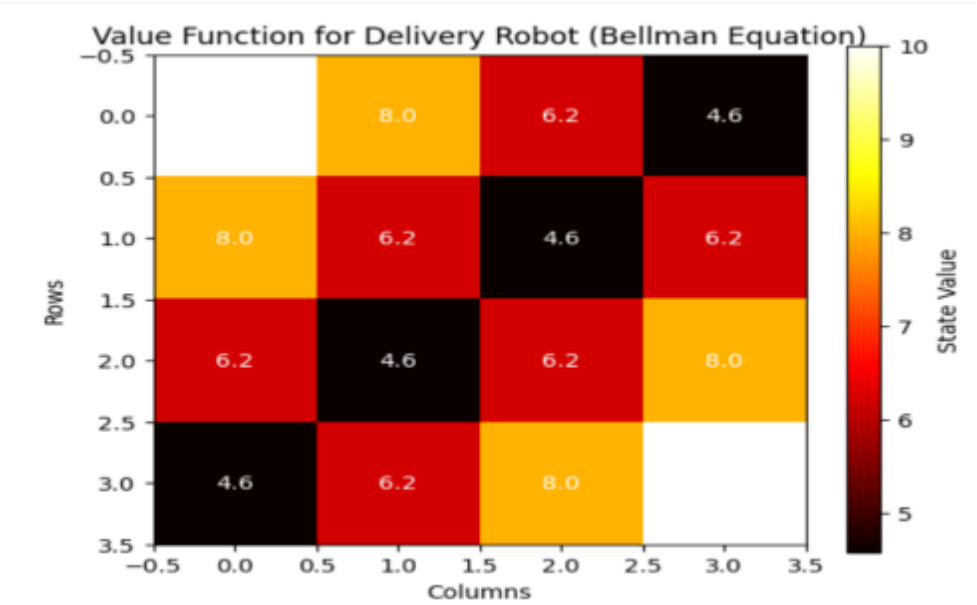
CODE:

```
import numpy as np
import matplotlib.pyplot as plt

# Warehouse setup (4x4 grid)
rows, cols = 4, 4
states = [(i, j) for i in range(rows) for j in range(cols)]

# Delivery points (goal states)
delivery_points = [(0, 0), (3, 3)]

# Parameters
gamma = 0.9
theta = 1e-4  # convergence threshold

# Rewards
def reward(state):
    if state in delivery_points:
        return 10
    return -1

# Next possible moves
def next_states(state):
    i, j = state
    moves = []

    if i > 0: moves.append((i-1, j))   # up
    if i < rows-1: moves.append((i+1, j))  # down
    if j > 0: moves.append((i, j-1))   # left
    if j < cols-1: moves.append((i, j+1))  # right

    return moves
```

```
# Initialize value function
V = {s: 0 for s in states}

# ---- VALUE ITERATION ----
while True:
    delta = 0
    new_V = V.copy()

    for s in states:
        if s in delivery_points:
            new_V[s] = reward(s)  # terminal reward
            continue

        values = []
        for next_s in next_states(s):
            val = reward(s) + gamma * V[next_s]
            values.append(val)

        new_V[s] = max(values)
        delta = max(delta, abs(new_V[s] - V[s]))

    V = new_V
    if delta < theta:
        break

# Convert V into 2D grid for visualization
value_grid = np.zeros((rows, cols))
for (i, j), v in V.items():
    value_grid[i, j] = v

# ---- VISUALIZATION ----
plt.figure(figsize=(6, 5))
plt.imshow(value_grid, cmap="hot", interpolation="nearest")
plt.colorbar(label="State Value")
plt.title("Value Function for Delivery Robot (Bellman Equation)")
plt.xlabel("Columns")
plt.ylabel("Rows")
```

```
# Display values on grid cells
for i in range(rows):
    for j in range(cols):
        plt.text(j, i, f"{value_grid[i, j]:.1f}",
                 ha="center", va="center", color="white")

plt.show()
```

OUTPUT:



EXPERIMENT 8:

Simulate an autonomous car navigating a simple road network with intersections. Design policies for the car to follow traffic rules and reach the destination safely. Implement these policies in Python and evaluate their effectiveness (success rate, average steps, collisions).

CODE:

```
# Autonomous car simulation (simple grid + intersections + traffic lights)
# Policies: FollowLights (obeys traffic lights), Aggressive (ignores lights), Cautious (waits extra)
# Evaluation: success rate, avg steps to reach, collisions

import numpy as np
import random

random.seed(42)
np.random.seed(42)

# Grid / environment parameters
ROWS, COLS = 5, 5
START = (4, 0)
GOAL = (0, 4)
OBSTACLES = []   # no static obstacles for simplicity; intersections only
INTERSECTIONS = {(2,2), (1,3), (3,1)}  # intersections with traffic lights
LIGHT_PERIOD = 4  # light flips every 4 time-steps: 0..1..2..3 then flips
MAX_STEPS = 40

ACTIONS = {'U':(-1,0),'D':(1,0),'L':(0,-1),'R':(0,1)}
ACTION_LIST = list(ACTIONS.keys())

def valid(pos):
    r,c = pos
    return 0 <= r < ROWS and 0 <= c < COLS

def step(pos, action):
    dr,dc = ACTIONS[action]
    new = (pos[0]+dr, pos[1]+dc)
    if not valid(new):
        return pos, False  # invalid move: stays, counts as collision-like (we'll treat separately)
```

```
    return new, True


def light_state(step_count, intersection):
    # simple round-robin: for each intersection define phase offset to vary patterns
    offsets = { (2,2):0, (1,3):1, (3,1):2 }
    phase = ((step_count + offsets[intersection]) // LIGHT_PERIOD) % 2
    # phase 0 -> NS green (allow Up/Down), phase 1 -> EW green (allow Left/Right)
    return 'NS' if phase == 0 else 'EW'


# Policies
def policy_follow_lights(state, step_count):
    pos = state
    # simple short-path policy: greedy toward goal (manhattan)
    # prefer horizontal or vertical based on distance
    dr = GOAL[0] - pos[0]
    dc = GOAL[1] - pos[1]
    # choose action list by priority
    pri = []
    if abs(dr) >= abs(dc):
        pri += ['D' if dr>0 else 'U']
        if dc!=0: pri += ['R' if dc>0 else 'L']
    else:
        pri += ['R' if dc>0 else 'L']
        if dr!=0: pri += ['D' if dr>0 else 'U']
    pri += [a for a in ACTION_LIST if a not in pri]
    for a in pri:
        newpos, valid_move = step(pos, a)
        if not valid_move:
            continue
        # if moving through intersection, check light
        if newpos in INTERSECTIONS:
            ls = light_state(step_count, newpos)
            # if moving Up/Down allow only NS; Left/Right allow only EW
            if a in ('U','D') and ls != 'NS':
                continue  # wait instead of moving
            if a in ('L','R') and ls != 'EW':
                continue
        # allowed
```

```
            return a
    # if none allowed, wait (return None)
    return None

def policy_aggressive(state, step_count):
    pos = state
    dr = GOAL[0] - pos[0]
    dc = GOAL[1] - pos[1]
    # same greedy priorities but ignore lights: always move
    pri = []
    if abs(dr) >= abs(dc):
        pri += ['D' if dr>0 else 'U']
        if dc!=0: pri += ['R' if dc>0 else 'L']
    else:
        pri += ['R' if dc>0 else 'L']
        if dr!=0: pri += ['D' if dr>0 else 'U']
    pri += [a for a in ACTION_LIST if a not in pri]
    for a in pri:
        newpos, valid_move = step(pos, a)
        if not valid_move:
            continue
        return a
    return None

def policy_cautious(state, step_count):
    # like follow_lights but also waits one extra step at intersections even if green (simulates cautious
behavior)
    pos = state
    # if currently at intersection, wait one step to "scan"
    if pos in INTERSECTIONS and (step_count % 3 == 0):
        return None
    return policy_follow_lights(state, step_count)

# Simulator
def run_episode(policy_fn, render=False):
    pos = START
    steps = 0
    collisions = 0
```

```
    for t in range(MAX_STEPS):
        if pos == GOAL:
            return True, steps, collisions
        action = policy_fn(pos, t)
        if action is None:
            steps += 1
            continue
        newpos, moved = step(pos, action)
        # moving into invalid cell treated as collision
        if not moved:
            collisions += 1
            steps += 1
            continue
        # if moving into intersection while light forbids (only possible for aggressive), treat as collision
        if newpos in INTERSECTIONS:
            ls = light_state(t, newpos)
            if policy_fn is policy_aggressive:
                # aggressive ignores lights: penalize if movement conflict with light
                if action in ('U','D') and ls != 'NS':
                    collisions += 1
                if action in ('L','R') and ls != 'EW':
                    collisions += 1
        pos = newpos
        steps += 1
    # max steps reached, treat as failure
    return False, steps, collisions

def evaluate(policy_fn, episodes=200):
    success = 0
    total_steps = 0
    total_coll = 0
    for _ in range(episodes):
        s, st, coll = run_episode(policy_fn)
        if s: success += 1
        total_steps += st
        total_coll += coll
    return {
        'success_rate': success/episodes,
```

```
        'avg_steps': total_steps/episodes,
        'avg_collisions': total_coll/episodes
    }

# Evaluate all three policies
policies = {
    'FollowLights': policy_follow_lights,
    'Aggressive': policy_aggressive,
    'Cautious': policy_cautious
}

results = {}
for name, fn in policies.items():
    res = evaluate(fn, episodes=500)
    results[name] = res

# Print concise results
for name, r in results.items():
    print(f"{name}: Success={r['success_rate']*100:.1f}%, AvgSteps={r['avg_steps']:.2f},
AvgCollisions={r['avg_collisions']:.3f}")
```

OUTPUT:

```
FollowLights: Success=100.0%, AvgSteps=8.00, AvgCollisions=0.000
Aggressive: Success=100.0%, AvgSteps=8.00, AvgCollisions=2.000
Cautious: Success=100.0%, AvgSteps=9.00, AvgCollisions=0.000
```

EXPERIMENT 9:

A call centre wants to optimize the assignment of customer service representatives (CSRs) to incoming customer calls. Implement a Monte Carlo simulation to estimate the value function for different CSR assignment policies (Random Policy, Greedy Policy, and Priority Policy) in Python.

CODE:

```
import numpy as np
import random

random.seed(42)

# ------------------------------
# Environment Setup
# ------------------------------

# Each CSR has a skill level (0 to 1)
CSRs = {
    "CSR1": 0.9,
    "CSR2": 0.7,
    "CSR3": 0.5
}

# Calls have difficulty levels (0 to 1)
def generate_call():
    return round(random.uniform(0.1, 1.0), 2)

# Reward: Higher reward if CSR skill >= call difficulty
def get_reward(skill, difficulty):
    if skill >= difficulty:
        return 5 - (skill - difficulty)  # smaller gap = better handling
    else:
        return -2  # failure case



# ------------------------------
# Policies
```

```python
# ----------------------------
def random_policy(call_difficulty):
    return random.choice(list(CSRs.keys()))

def greedy_policy(call_difficulty):
    # Choose CSR with the highest skill
    return max(CSRs, key=lambda x: CSRs[x])

def priority_policy(call_difficulty):
    # Assign tough calls to best CSR, easy calls to lowest skill CSR
    if call_difficulty > 0.7:
        return "CSR1"   # best
    elif call_difficulty > 0.4:
        return "CSR2"
    else:
        return "CSR3"



# ----------------------------
# Monte Carlo Simulation
# ----------------------------
def monte_carlo(policy_fn, episodes=1000):
    total_reward = 0

    for _ in range(episodes):
        call = generate_call()
        assigned = policy_fn(call)
        reward = get_reward(CSRs[assigned], call)
        total_reward += reward

    return total_reward / episodes  # value estimate



# ----------------------------
# Evaluate All Policies
# ----------------------------
policies = {
```

```
    "Random Policy": random_policy,
    "Greedy Policy": greedy_policy,
    "Priority Policy": priority_policy
}

results = {}

for name, fn in policies.items():
    value = monte_carlo(fn)
    results[name] = value

# Output Results
print ("Monte Carlo Value Function Estimates:\n")
for name, val in results. Items ():
    print(f"{name}: {val:.3f}")
```

OUTPUT:

```
Monte Carlo Value Function Estimates:

Random Policy: 2.470
Greedy Policy: 3.964
Priority Policy: 4.188
```

EXPERIMENT 10:

A financial institution wants to optimize its investment strategy. Use a basic Policy Gradient method to simulate and optimize the investment policy for maximum returns. Implement this in Python.

CODE:

```
import numpy as np

# Investment options: (expected reward)
investment_returns = {
    "Low": 1.0,     # safe, small profit
    "Medium": 2.0,  # moderate risk
    "High": 4.0     # high profit, high risk
}

actions = list(investment_returns.keys())

# Initial policy probabilities (uniform random start)
policy = np.array([1/3, 1/3, 1/3], dtype=float)

lr = 0.01          # learning rate
episodes = 5000

def choose_action(policy):
    return np.random.choice(actions, p=policy)

def get_reward(action):
    base = investment_returns[action]

    # risky investments fluctuate
    if action == "High":
        return np.random.normal(base, 2)
    elif action == "Medium":
        return np.random.normal(base, 1)
    else:
        return np.random.normal(base, 0.5)
```

```
for episode in range(episodes):

    action = choose_action(policy)
    reward = get_reward(action)

    # policy gradient update (REINFORCE)
    action_idx = actions.index(action)

    grad = np.zeros_like(policy)
    grad[action_idx] = 1 - policy[action_idx]

    policy += lr * reward * grad
    policy = np.maximum(policy, 0.0001)      # avoid negatives
    policy /= np.sum(policy)                 # normalize

print("\nOptimized Investment Policy:")
for action, prob in zip(actions, policy):
    print(f"{action}: {prob:.3f}")
```

OUTPUT:

```
Optimized Investment Policy:
Low: 0.000
Medium: 0.304
High: 0.696
```